BÀI THỰC HÀNH

LAB 09

Đo khoảng cách với HC-SR04

Đọc khoảng cách bằng Ultrasonic Device API với EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu . . . 2
1.1 Tổng quan bài lab . . . 2
1.2 Mục tiêu . . . 2
2 Kiến thức nền . . . 2
2.1 Đo khoảng cách bằng sóng siêu âm . . . 2
2.2 Tín hiệu TRIG và ECHO . . . 2
2.3 Từ thời gian ECHO đến khoảng cách . . . 3
3 Thiết lập phần cứng . . . 3
3.1 Phần cứng sử dụng . . . 3
3.2 Ánh xạ chân sử dụng . . . 3
3.3 Kết nối HC-SR04 . . . 4
4 EduFramework API . . . 4
4.1 `ultrasonicBegin()` . . . 4
4.2 `ultrasonicRead()` . . . 5
5 Bài thực hành . . . 5
5.1 Yêu cầu . . . 5
5.2 Chương trình . . . 5
5.3 Kiểm chứng . . . 6
6 Mở rộng . . . 7
6.1 Đọc khoảng cách theo inch với `ultrasonicReadInch()` . . . 7
6.2 Lọc kết quả với `ultrasonicReadFiltered()` . . . 7
6.3 Một số API Ultrasonic khác . . . 8
7 Tài liệu tham khảo . . . 8

1 Giới thiệu

1.1 Tổng quan bài lab

Cảm biến siêu âm có thể xác định khoảng cách tới vật thể mà không cần tiếp xúc trực tiếp. HC-SR04 thực hiện phép đo bằng cách phát tín hiệu siêu âm, chờ tín hiệu phản xạ trở lại và sử dụng thời gian truyền của tín hiệu để suy ra khoảng cách [1] .

Trong EduFramework, quá trình tạo tín hiệu kích hoạt, đo thời gian phản hồi và chuyển đổi kết quả thành khoảng cách được đóng gói trong Ultrasonic Device API. Ứng dụng vì vậy có thể làm việc trực tiếp với giá trị khoảng cách theo centimet thay vì tự xử lý từng bước của quá trình đo [6] .

Bài thực hành sử dụng `ultrasonicBegin()` để khởi tạo HC-SR04 và `ultrasonicRead()` để đọc khoảng cách, sau đó hiển thị kết quả trên Serial Monitor. Các phép đọc theo đơn vị khác, lọc nhiều mẫu và cấu hình thời gian chờ được dành cho phần Mở rộng.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả nguyên lý cơ bản của phép đo khoảng cách bằng sóng siêu âm.
2. Giải thích vai trò của hai tín hiệu `TRIG` và `ECHO` trên HC-SR04.
3. Giải thích quan hệ giữa thời gian phản hồi của tín hiệu siêu âm và khoảng cách tới vật thể.
4. Sử dụng `ultrasonicBegin()` và `ultrasonicRead()` để đo khoảng cách theo centimet.
5. Xây dựng và kiểm chứng ứng dụng giám sát khoảng cách bằng Serial Monitor.

2 Kiến thức nền

2.1 Đo khoảng cách bằng sóng siêu âm

HC-SR04 sử dụng sóng siêu âm để đo khoảng cách tới vật thể. Khi bắt đầu một phép đo, module phát tín hiệu siêu âm khoảng `40 kHz` . Tín hiệu truyền trong không khí, phản xạ tại vật thể và quay trở lại cảm biến. Khoảng thời gian từ khi phát đến khi nhận phản xạ được sử dụng để xác định khoảng cách [1] [2] .

Quá trình đo có thể mô tả theo luồng:

`Cảm biến` → `Sóng siêu âm` → `Vật thể` → `Sóng phản xạ` → `Cảm biến`

Do tín hiệu phải truyền từ cảm biến tới vật thể rồi quay trở lại, thời gian đo được tương ứng với quãng đường hai chiều chứ không chỉ khoảng cách một chiều từ cảm biến tới vật thể [3] .

2.2 Tín hiệu TRIG và ECHO

HC-SR04 sử dụng hai tín hiệu số chính trong quá trình đo. `TRIG` là đầu vào dùng để bắt đầu phép đo, còn `ECHO` là đầu ra biểu diễn thời gian phản hồi của tín hiệu siêu âm. Một xung mức `HIGH` ít nhất khoảng `10 µs` tại `TRIG` kích hoạt module phát một chuỗi tám chu kỳ siêu âm `40 kHz` [3] [2] .

Sau khi phát tín hiệu, độ rộng xung mức `HIGH` tại `ECHO` thay đổi theo thời gian truyền và phản xạ của sóng siêu âm. Vì vậy, đo khoảng thời gian `ECHO` ở mức `HIGH` cung cấp thông tin cần thiết để tính khoảng cách [1] [3] .

Trong Ultrasonic Device của EduFramework, `TRIG` được cấu hình là Digital Output và `ECHO` được cấu hình là Digital Input. Device API thực hiện tuần tự quá trình phát xung kích hoạt, chờ `ECHO`, đo độ rộng xung và xử lý kết quả [6].

2.3 Từ thời gian ECHO đến khoảng cách

Gọi t là thời gian sóng siêu âm đi từ cảm biến tới vật thể rồi quay trở lại và v là vận tốc truyền âm, khoảng cách một chiều d được xác định theo:

$$d = \frac{v \times t}{2}$$

Phép chia cho `2` xuất hiện vì thời gian đo bao gồm cả hành trình đi và hành trình quay về của tín hiệu. HC-SR04 có thể quy đổi thời gian `ECHO` theo micro giây sang khoảng cách bằng quan hệ xấp xỉ `µs / 58 = cm` [3]. Phần triển khai hiện tại của EduFramework sử dụng hệ số `0.017` để chuyển độ rộng xung `ECHO` tính bằng micro giây thành khoảng cách tính bằng centimet: `distance_cm = duration_us × 0.017`
Giá trị này tương ứng với phép quy đổi khoảng cách dựa trên thời gian truyền hai chiều của sóng siêu âm [3] [6].
Thư viện thiết bị siêu âm còn sử dụng cơ chế giới hạn thời gian chờ để tránh chương trình chờ vô hạn khi không nhận được tín hiệu `ECHO` hợp lệ. Với cấu hình mặc định, thời gian chờ là `30000 µs`; khi phép đo không hợp lệ, API đọc khoảng cách trả về giá trị lỗi `ULTRASONIC_INVALID_DISTANCE_CM` [6].

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng MaaZEDU Development Board và module HC-SR04. Module có bốn chân `VCC`, `GND`, `TRIG` và `ECHO`; trong đó `TRIG` và `ECHO` được kết nối tới các Logical Pin GPIO của EduFramework [1] [6].

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
HC-SR04 ultrasonic sensor	Module cảm biến siêu âm dùng để đo khoảng cách tới vật thể.
Jumper wires	Kết nối nguồn và các tín hiệu `TRIG`, `ECHO` giữa HC-SR04 và MaaZEDU.
USB Cable	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 9.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân sử dụng

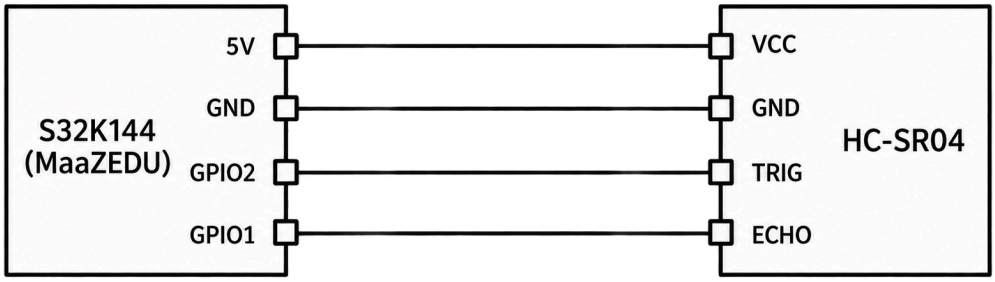
Bài thực hành sử dụng `GPIO2` cho tín hiệu `TRIG` và `GPIO1` cho tín hiệu `ECHO`. Trên MaaZEDU Development Board, `GPIO2` ánh xạ tới `PTD14` và `GPIO1` ánh xạ tới `PTD17` [5].

Thành phần	Logical Pin	MCU Pin	Chức năng
HC-SR04 `TRIG`	`GPIO2`	`PTD14`	Digital Output
HC-SR04 `ECHO`	`GPIO1`	`PTD17`	Digital Input

Bảng 9.3.2. Ánh xạ tín hiệu HC-SR04 sử dụng trong bài thực hành

3.3 Kết nối HC-SR04

Trong cấu hình phần cứng đã được kiểm chứng cho bài thực hành, HC-SR04 được cấp nguồn từ `5V`, sử dụng chung `GND` với MaaZEDU, `TRIG` được nối tới `GPIO2` và `ECHO` được nối tới `GPIO1`. HC-SR04 sử dụng nguồn `5V` trong cấu hình hoạt động được mô tả cho module [1] [2].



Hình 9.3.1. Sơ đồ kết nối HC-SR04 với MaaZEDU Development Board

Ghi chú

Bài thực hành sử dụng các Logical Pin `GPIO2` và `GPIO1` thay vì thao tác trực tiếp với tên chân vi điều khiển. Cách sử dụng này giữ chương trình ở tầng API của EduFramework và phù hợp với ánh xạ chân của MaaZEDU [5] [6].

4 EduFramework API

Bài thực hành chính chỉ cần hai API của Ultrasonic Device: `ultrasonicBegin()` và `ultrasonicRead()`. Các phép đọc thời gian, đổi đơn vị, lọc nhiều mẫu và cấu hình timeout được dành cho phần Mở rộng [6].

4.1 `ultrasonicBegin()`

`ultrasonicBegin()` khởi tạo cảm biến siêu âm mặc định với hai Logical Pin dùng cho `TRIG` và `ECHO`. API cấu hình chân `TRIG` làm đầu ra, chân `ECHO` làm đầu vào và sử dụng timeout mặc định của Ultrasonic Device [6].

`ultrasonicBegin`

Khởi tạo cảm biến HC-SR04 mặc định và cấu hình hai chân `TRIG`, `ECHO`.

Cú pháp

```
ultrasonicBegin(trigPin, echoPin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`trigPin`	Digital Logical Pin	Chân digital nối với `TRIG` của HC-SR04.

Tham số	Kiểu / giá trị chấp nhận	Mô tả
echoPin	Digital Logical Pin	Chân digital nối với `ECHO` của HC-SR04.

Giá trị trả về

Không có giá trị trả về.

Ví dụ:

```
ultrasonicBegin(GPIO2, GPIO1);
```

4.2 `ultrasonicRead()`

`ultrasonicRead()` thực hiện một phép đo với cảm biến mặc định và trả về khoảng cách theo centimet. Nếu không đo được xung `ECHO` hợp lệ trước khi timeout, API trả về `ULTRASONIC_INVALID_DISTANCE_CM`, có giá trị `-1.0F` [6].

`ultrasonicRead`

Thực hiện phép đo với HC-SR04 và trả về khoảng cách theo centimet.

Cú pháp

```
ultrasonicRead();
```

Giá trị trả về

Khoảng cách theo centimet khi phép đo thành công; `ULTRASONIC_INVALID_DISTANCE_CM` khi timeout hoặc phép đo không hợp lệ.

Ví dụ:

```
float distance = ultrasonicRead();
```

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình đo khoảng cách bằng HC-SR04 với `TRIG` nối tới `GPIO2` và `ECHO` nối tới `GPIO1`. Chương trình khởi tạo `Serial1` ở baud rate `9600`, đọc khoảng cách theo centimet và cập nhật kết quả trên Serial Monitor sau mỗi `500 ms`.

Nếu phép đo không hợp lệ, chương trình hiển thị `Distance: Timeout`. Sau khi chương trình hoạt động ổn định, thay đổi khoảng cách giữa cảm biến và một vật phẳng để quan sát sự thay đổi của kết quả.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"
#include "ultrasonic.h"

int main(void)
{
    float distance = ULTRASONIC_INVALID_DISTANCE_CM;

    setup();

    Serial1_begin(9600U);
    ultrasonicBegin(GPIO2, GPIO1);

    Serial1_println("HC-SR04 Distance Monitor");

    while (1)
    {
        distance = ultrasonicRead();

        if (ULTRASONIC_INVALID_DISTANCE_CM != distance)
        {
            Serial1_print("Distance: ");
            Serial1_printFloat(distance);
            Serial1_println(" cm");
        }
        else
        {
            Serial1_println("Distance: Timeout");
        }

        delay(500U);
    }

    return 0;
}
```

Mã nguồn 9.5.1. Chương trình đo khoảng cách bằng HC-SR04

`setup()` khởi tạo các thành phần nền tảng của EduFramework. `Serial1_begin()` khởi tạo kênh Serial Monitor, còn `ultrasonicBegin(GPIO2, GPIO1)` khởi tạo cảm biến với `GPIO2` là `TRIG` và `GPIO1` là `ECHO` [6]. Trong vòng lặp chính, `ultrasonicRead()` thực hiện quá trình kích hoạt cảm biến, đo thời gian `ECHO` và chuyển đổi kết quả thành centimet. Chương trình kiểm tra giá trị trả về trước khi in kết quả để phân biệt phép đo hợp lệ với trường hợp timeout [6].

Luồng xử lý của ứng dụng có thể tóm tắt:

`Vật thể` → `HC-SR04` → `Thời gian ECHO` → `ultrasonicRead()` → `Khoảng cách` → `Serial1` → `Serial Monitor` [6].

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó mở Serial Monitor với baud rate `9600`. Đặt một vật phẳng phía trước HC-SR04 và quan sát giá trị khoảng cách. Di chuyển vật ra xa cảm biến rồi đưa lại gần để kiểm tra xu hướng thay đổi của kết quả.

Dữ liệu trên Serial Monitor có dạng:

```
HC-SR04 Distance Monitor
Distance: 10.24 cm
```

```
Distance: 10.19 cm
Distance: 20.36 cm
Distance: 30.41 cm
```

Khi không nhận được phép đo hợp lệ trong thời gian chờ, chương trình hiển thị:

```
Distance: Timeout
```

KẾT QUẢ MONG ĐỢI

Serial Monitor cập nhật kết quả khoảng mỗi `500 ms`. Khi vật thể được đưa ra xa cảm biến, giá trị khoảng cách có xu hướng tăng; khi vật thể được đưa lại gần, giá trị có xu hướng giảm. Nếu không thu được tín hiệu `ECHO` hợp lệ trước khi timeout, chương trình hiển thị trạng thái `Timeout`. Hành vi này phù hợp với nguyên lý đo thời gian phản hồi của HC-SR04 và cơ chế xử lý của Ultrasonic Device [1] [6].

6 Mở rộng

Bài thực hành chính chỉ sử dụng phép đọc khoảng cách theo centimet. Ultrasonic Device còn cung cấp các API để đổi đơn vị, quan sát thời gian phản hồi, thay đổi timeout và xử lý nhiều mẫu đo [6].

6.1 Đọc khoảng cách theo inch với `ultrasonicReadInch()`

`ultrasonicReadInch()` thực hiện phép đo với cảm biến mặc định và trả về khoảng cách theo inch. API sử dụng kết quả đo khoảng cách và chuyển đổi đơn vị bên trong Device [6].

`ultrasonicReadInch`

Đọc khoảng cách từ HC-SR04 và trả về kết quả theo inch.

Cú pháp

```
ultrasonicReadInch();
```

Giá trị trả về

Khoảng cách theo inch khi phép đo thành công; `ULTRASONIC_INVALID_DISTANCE_CM` khi phép đo không hợp lệ.

Ví dụ:

```
float distanceInch = ultrasonicReadInch();
```

Mở rộng: Sửa chương trình để hiển thị khoảng cách theo cả centimet và inch trên Serial Monitor.
Kết quả có thể trình bày theo dạng:

```
Distance: 25.40 cm
Distance: 10.00 inch
```

6.2 Lọc kết quả với `ultrasonicReadFiltered()`

Một phép đo đơn có thể thay đổi giữa các lần đọc. `ultrasonicReadFiltered()` thu nhiều mẫu khoảng cách trước khi trả về kết quả đã xử lý. Với cấu hình mặc định hiện tại, Device thu `20` mẫu, sắp xếp các mẫu, loại `5` mẫu ở đầu thấp và `5` mẫu ở đầu cao, sau đó lấy trung bình các mẫu còn lại [6].

`ultrasonicReadFiltered`

Thu nhiều mẫu khoảng cách và trả về giá trị đã được xử lý từ nhóm mẫu ở giữa.

Cú pháp

```
ultrasonicReadFiltered();
```

Giá trị trả về

Khoảng cách đã lọc theo centimet khi có dữ liệu hợp lệ; `ULTRASONIC_INVALID_DISTANCE_CM` nếu không thu được kết quả hợp lệ.

Ví dụ:

```
float filteredDistance = ultrasonicReadFiltered();
```

Mở rộng: Giữ vật thể ở một vị trí cố định và so sánh kết quả của `ultrasonicRead()` với `ultrasonicReadFiltered()` trên Serial Monitor.
Ví dụ:

```
Raw: 25.36 cm
Filtered: 25.18 cm
```

6.3 Một số API Ultrasonic khác

EduFramework còn cung cấp một số API bổ sung [6] :

- `ultrasonicReadDuration()` trả về độ rộng xung `ECHO` theo microsecond.
- `ultrasonicSetTimeout(timeoutUs)` thay đổi thời gian chờ của phép đo mặc định.
- Nhóm API `Ultrasonic_Begin()`, `Ultrasonic_ReadCm()`, `Ultrasonic_ReadInch()` và `Ultrasonic_ReadCmFiltered()` sử dụng đối tượng `Ultrasonic_t` để hỗ trợ nhiều instance cảm biến trong cùng ứng dụng.

Các API trên không cần thiết cho bài thực hành chính. Chúng phù hợp khi cần quan sát trực tiếp thời gian phản hồi, điều chỉnh timeout hoặc sử dụng nhiều cảm biến siêu âm trong một hệ thống [6] .
Mở rộng: Sử dụng `ultrasonicReadDuration()` để hiển thị đồng thời thời gian `ECHO` và khoảng cách, sau đó quan sát cách hai giá trị thay đổi khi vật thể được di chuyển.

7 Tài liệu tham khảo

[1] New York University, *Lab: Ultrasonic Distance Sensor* , ITP Physical Computing.

[2] Oregon State University, *Sonar Rangefinder HC-SR04* , TekBots.

[3] SparkFun Electronics, *HC-SR04 Ultrasonic Sensor Datasheet* , Technical Datasheet.

[4] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[5] MaaZEDU, *MaaZEDU Development Board Guide* .

[6] EduFramework, *Ultrasonic Device API* , EduFramework Source Code.